

ROS2

DAY 1 — MANUAL 2

(ROS2 PRACTICAL BEGINNER GUIDE – PYTHON)

Complete Beginner-Friendly

Chapter 1 — Introduction

1.1 Purpose of Manual 2

Manual 2 provides a complete, practical introduction to ROS2 Python programming. It teaches the minimum required steps to create:

- a workspace
- a package
- a node

This document focuses on hands-on implementation.

1.2 Learning Outcomes

After studying this manual, the learner will be able to:

1. Create and manage a ROS2 workspace.
 2. Create a Python package using the ROS2 CLI.
 3. Create a node manually inside the package.
 4. Register the node using `setup.py`.
 5. Build and run ROS2 applications with `colcon`.
-

1.3 ROS2 Python Programming Overview

ROS2 Python development relies on:

- `rclpy` (ROS client library for Python)

- ament_python (Python build system for ROS2)
- Python scripts acting as nodes
- entry points used to expose nodes as executables

ROS2 separates code into packages, grouped inside workspaces.

Chapter 2 — ROS2 Workspace

2.1 Definition of Workspace

A workspace is a directory in which all ROS2 packages and associated build outputs are stored.

Typical workspace structure:

```
ros2_ws/  
├── src/  
├── build/  
├── install/  
└── log/
```

2.2 Importance of Workspace

A workspace allows ROS2 to:

- organize multiple packages
 - compile all packages together
 - maintain clean separation between source, build, and installed files
-

2.3 Workspace Creation Procedure

Execute the following commands:

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws
```

The workspace is now ready to contain packages.

Chapter 3 — Python Package Creation

3.1 Definition of a ROS2 Package

A ROS2 package is a self-contained unit that contains:

- source code
- metadata
- configuration
- dependencies

It is the fundamental building block of any ROS2 project.

3.2 Standard Structure of a Python ROS2 Package

After creation, the package structure appears as:

```
my_py_pkg/  
├── package.xml  
├── setup.py  
├── setup.cfg  
├── resource/  
└── my_py_pkg/  
    └── __init__.py
```

3.3 Creating a Python Package

Navigate to the src directory:

```
cd ~/ros2_ws/src
```

Create the package:

```
ros2 pkg create my_py_pkg --build-type ament_python
```

The folder `my_py_pkg` will be generated automatically.

3.4 Understanding Auto-Generated Files

package.xml

Contains package metadata and dependency declarations.

setup.py

Defines how Python nodes are installed.
This file must contain entry points for ROS2 executables.

setup.cfg

Installation configuration file for Python packaging.

my_py_pkg directory

Contains Python modules and scripts.

Chapter 4 — Manual Node Creation

4.1 Definition of a Node

A node is an executable that performs a specific task in a ROS2 system.
Examples:

- sensor reader
- motor controller
- publisher
- subscriber

ROS2 systems are composed of many small, independent nodes.

4.2 Requirements of a Python Node

A valid Python ROS2 node must:

1. Import `rclpy`.
2. Create a class that inherits from `Node`.
3. Have a `main()` function.

4. Use `rclpy.spin()` to keep the node running.
5. Be registered in `setup.py`.

4.3 Writing the Node

Navigate to the package's Python directory:

```
cd ~/ros2_ws/src/my_py_pkg/my_py_pkg
```

Create the Python file:

```
touch simple_node.py
```

Write the following code:

```
import rclpy
from rclpy.node import Node

class SimpleNode(Node):
    def __init__(self):
        super().__init__('simple_node')
        self.get_logger().info("Python node is running.")

def main(args=None):
    rclpy.init(args=args)
    node = SimpleNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

4.4 Register the Node in setup.py

Open `setup.py` and locate the section named `entry_points`.

Modify it to:

```
entry_points={
    'console_scripts': [
        'simple_node = my_py_pkg.simple_node:main',
    ],
},
```

This registers the node so that it can be launched using the ROS2 CLI.

Chapter 5 — Building and Running

5.1 Definition of colcon

colcon (Collective Construction) is the ROS2 build tool. It builds all packages in the workspace and organizes build outputs.

5.2 Building the Workspace

Navigate to the workspace root:

```
cd ~/ros2_ws
```

Build:

```
colcon build
```

5.3 Sourcing the Environment

Before running packages, source the environment:

```
source install/setup.bash
```

This ensures the system recognizes installed ROS2 packages.

5.4 Running the Node

Run the node using ROS2 CLI:

```
ros2 run my_py_pkg simple_node
```

Expected output:

```
[INFO] [simple_node]: Python node is running.
```
